



KHUFU

FREE CHECKLIST

47 questions to ask before you hire anyone to build your SaaS

The vetting checklist for founders who cannot read code — what to ask, why it matters, and the answer that should end the conversation.

What this is

The hardest part of hiring someone to build software is that you cannot inspect the product until you have already paid for most of it. By the time bad work becomes visible, you have spent the budget and the calendar.

The way around that is not technical knowledge. It is asking questions whose answers are hard to fake — and knowing what a good answer sounds like before you hear one.

These are the 47 I would ask. They are uncomfortable, including for me. If a provider gets defensive at question 14 or 20, that is the checklist working — that is exactly what it costs to find out cheaply.

Who this is for

Non-technical founders about to hire an agency, a freelancer or a development team, who need to evaluate people whose work they cannot inspect.

What's inside

- 01 How to use this list**
Running 47 questions without turning a sales call into an interrogation.
- 02 The five answers that should end the conversation**
The responses that are not negotiating positions — they are exits.
- 03 Scope and understanding (1–6)**
Whether they understood the product, or just the feature list.
- 04 Price, contract and what happens when it slips (7–13)**
Where the money actually moves when things do not go to plan.
- 05 Ownership: code, infrastructure, accounts, IP (14–20)**
The section to read twice. Ownership is either concrete or it is marketing.
- 06 Who actually does the work (21–25)**
The gap between who sells the project and who builds it.
- 07 Technical choices and lock-in (26–31)**
Questions you can ask usefully even if you cannot evaluate the answer yourself.
- 08 Process and communication (32–36)**
How you find out something is wrong — early, rather than at the end.

09 Quality, testing and security (37–42)

The work you are paying for that you will never see — until it fails.

10 Launch, handover and life after (43–47)

What happens on the day the project ends, and the month after.

How to use this list

Running 47 questions without turning a sales call into an interrogation.

Do not read all 47 down a call. You will sound like a procurement form and you will learn less, because the useful signal is in how someone thinks, not in whether they have a prepared answer.

Pick twelve

Choose two or three from each section that map to your actual risk. If you are non-technical, weight ownership and quality. If you have a hard deadline, weight price and process. If the provider is a team rather than a person, weight who does the work.

Ask the open ones first

Start with question 1 — "describe my product back to me in one sentence." It is the highest-signal question in the list and it takes ten seconds. Someone who repeats your feature list has not understood the product. Someone who tells you what it is for, and what they would cut, has.

Listen for the shape of the answer, not the content

- Specifics beat reassurance. "Yes, we handle that" is not an answer; "here is how we handled it on X" is.
- Volunteered downsides are the strongest possible signal. Anyone who tells you unprompted what could go wrong has done this before.
- Watch for the pivot to sales. A question about ownership answered with a paragraph about their process is a no.
- A slow, honest "I do not know, I would need to check" beats a fast confident wrong answer every time.

Get the answers in writing

For anything in the ownership and price sections, ask for the answer by email afterwards. It is a reasonable request, it takes them two minutes, and a provider who will not put an ownership answer in writing has told you what you needed to know.

The five answers that should end the conversation

The responses that are not negotiating positions — they are exits.

Most bad answers are fixable with a better contract. These five are not, and you save yourself months by walking away the moment you hear one.

1. "We keep the code until final payment, then transfer it." Payment terms are fine; hostage terms are not. The repository should be in your organisation from day one, with commits landing in it as work happens. Anything else means a dispute leaves you with nothing.
2. "We use our own internal framework, it makes us faster." It makes them necessary. You will not be able to hire anyone else to maintain it, and that is the point of the arrangement.
3. "We can't tell you who will work on it until we start." You are buying a specific person's judgement. If they cannot name them, you are buying whoever is unallocated that month.
4. "We'll figure out the exact scope as we go." Without a written scope, there is no such thing as late and no such thing as out of scope. Time-and-materials work is legitimate — but only with a technical counterpart on your side who can tell you when to stop.
5. "Testing and deployment are a separate phase we can quote later." Software that is not deployed is not finished. If shipping is a separate line item, the quote you are comparing is not for a working product.

Scope and understanding (1–6)

Whether they understood the product, or just the feature list.

1 Describe my product back to me in one sentence.

Why The fastest test of whether someone listened. A person who understands the product can compress it; a person who does not will expand it.

Red flag Your feature list read back to you, or a sentence full of technology names rather than users.

2 What would you cut from this scope to make it fit the budget?

Why Everyone who has shipped has an opinion about what is not needed in a first version. It is the single clearest sign of experience.

Red flag "Nothing, we can do it all." That is a quote written to be accepted, not delivered.

3 What is the riskiest part of this build?

Why Forces them to have actually thought about your project instead of pattern-matching it to their last one.

Red flag No risk identified, or a generic answer about "communication" that would apply to any project.

4 What do you need from me, and by when?

Why Most delays are caused by the client, and a provider who has done this knows exactly which inputs stall a build — content, accounts, decisions, access.

Red flag "Nothing, we'll handle everything." Untrue, and it means you will be surprised on the day it becomes urgent.

5 What happens if my idea turns out to be wrong?

Why Tests whether they think in terms of your outcome or their invoice. The right answer involves building less, sooner, to find out.

Red flag Discomfort with the question, or a pitch for a bigger build to "cover all the cases".

6 Have you built something in this space before? What was hard about it?

Why The second half is the real question. Anyone can claim domain experience; only someone who lived it can name the specific difficulty.

Red flag A list of logos with no story attached, or a difficulty so generic it could be any project.

Price, contract and what happens when it slips (7–13)

Where the money actually moves when things do not go to plan.

7 Is this a fixed price or an estimate?

Why They are completely different products and the word "quote" hides the difference. A fixed price transfers risk to the builder; an estimate leaves it with you.

Red flag Ambiguity. If they will not say the words "fixed price", it is an estimate.

8 What specifically triggers a change order, and what does one cost?

Why Change orders are where a competitive quote becomes an expensive project. You want the threshold defined before you are emotionally committed.

Red flag "We'll be reasonable about it." Reasonable is not a rate, and it is never reasonable once the project is late.

9 If this takes twice as long as you estimated, who pays for the extra time?

Why The single most predictive question about how your project will end. Ask it directly and watch the pause.

Red flag Any version of "we'd have to discuss it". You are discussing it now, which is the only time you have leverage.

10 What is the payment schedule, and what is each payment tied to?

Why Payments tied to deliverables keep incentives aligned. Payments tied to dates pay for elapsed time whether or not anything shipped.

Red flag Large payment up front with nothing tied to delivery, or full payment before handover.

11 What is not included in this price?

Why The most useful question in the entire list. Hosting, third-party service fees, app store accounts, design, content, copywriting, and post-launch fixes are all commonly excluded.

Red flag "Everything is included." Nothing includes third-party fees. They are telling you they have not thought it through, or they are hoping you will not ask again.

12 What does support cost after launch, and what is the response time?

Why Software needs maintenance from the week it ships. If this is unpriced, you will negotiate it at your least powerful moment — when something is broken.

Red flag No answer, or support that only exists as an expensive retainer with no stated response time.

13 Can I stop the project halfway, and what do I get if I do?

Why Establishes whether partial work has value to you, or only to them. It also reveals the real ownership position faster than asking about ownership directly.

Red flag "You would get nothing" or a large termination fee. Work you paid for is work you should hold.

Ownership: code, infrastructure, accounts, IP (14–20)

The section to read twice. Ownership is either concrete or it is marketing.

14 Who owns the code, in writing, and from what moment?

Why In many jurisdictions the author holds copyright by default unless the contract assigns it. "You own it" in a sales call is not an assignment clause.

Red flag Ownership transferring only on final payment, or a licence to use rather than an assignment of ownership.

15 Whose accounts hold the hosting, the domain and the database?

Why Code ownership is worthless if your product runs on someone else's infrastructure account. This is the most common practical lock-in, and it is rarely malicious — it is just convenient for them.

Red flag Everything on the agency's accounts, billed through them, "for simplicity".

16 Do you use any proprietary framework, template or component library of your own?

Why A private framework means only they can maintain your product, and it will not be visible to you until you try to hire someone else.

Red flag Yes, combined with a licence rather than ownership of those parts. Open-source libraries are fine and normal; private ones are a trap.

17 Do I get the full commit history, or a final export?

Why History is how a new developer understands why the code is the way it is. A single "initial commit" containing everything destroys months of context and often hides who really wrote it.

Red flag A zip file, a single squashed commit, or "we'll push it at the end".

18 Who holds the API keys and secrets at the end of the project?

Why Credentials created under their accounts are credentials you cannot rotate, and a security exposure that outlives the relationship.

Red flag Keys created under personal accounts, or shared credentials in a document nobody can revoke.

19 What licences are the third-party components under?

Why Some open-source licences carry obligations that matter for a commercial product; some commercial components carry per-seat fees that arrive after launch.

Red flag They do not know. It is a two-minute check and a professional should have done it.

20 If I ended this relationship tomorrow, what would stop working?

Why The single best summary question in the list. The correct answer is "nothing, except further changes."

Red flag Anything else. Hosting, monitoring, a licence, an integration keyed to their account — each one is a dependency you did not know you were buying.

Who actually does the work (21–25)

The gap between who sells the project and who builds it.

21 Who exactly writes the code? Name them.

Why You are buying a specific person's judgement. In many agencies the person in the sales meeting is not the person who builds, and the gap between them is the whole risk.

Red flag Roles instead of names, or "one of our senior developers".

22 Will any part of this be subcontracted or offshored?

Why Not disqualifying — but it changes the communication chain, the time zones and the accountability, and you should know before rather than after.

Red flag Evasion. Find out now, or find out from a commit history later.

23 How many other projects will that person be on during mine?

Why A developer split across four projects is not giving you a quarter of their attention — context switching costs more than that.

Red flag No answer, or a number above two.

24 Can I see code that this person wrote themselves?

Why Even if you cannot read it, you can have someone else read it, and the willingness to show it is signal on its own.

Red flag Only polished case studies and screenshots. Every developer has something they can show.

25 What happens if that person is ill, or leaves mid-project?

Why A real risk with solo operators and small teams, and one worth pricing rather than pretending away.

Red flag No plan at all — or, from a solo builder, refusing to acknowledge it is a risk you are taking.

Technical choices and lock-in (26–31)

Questions you can ask usefully even if you cannot evaluate the answer yourself.

26 What stack are you using, and why that one for my case?

Why The second half matters. A good answer connects a technical choice to something specific about your product; a weak one describes what they always use.

Red flag A stack chosen because it is what they know, presented as if it were chosen for you.

27 Is this real code, or a no-code or low-code builder?

Why Both are legitimate, and they carry completely different futures. You need to know which one you are buying, and it is not always volunteered.

Red flag Vagueness about the platform, or a price that only makes sense if it is no-code while the pitch implies custom software.

28 If usage were a hundred times larger than expected, what breaks first?

Why A genuinely experienced builder knows the answer and will name a specific bottleneck. It also reveals whether they have run anything in production.

Red flag "It scales" with no specifics. Everything scales until a particular thing does not.

29 What third-party services will I depend on, and what do they cost monthly?

Why Your running costs are decided during the build, by someone who is not paying them.

Red flag No list, or no idea of the monthly total. Ask for the figure at your expected usage and at ten times that.

30 How hard would it be for another developer to take this over?

Why You are asking them to argue against their own indispensability, which is exactly why the answer is informative.

Red flag Discomfort, or an answer implying it would be difficult. A good builder is proud that it would be easy.

31 What decisions are you making now that would be expensive to reverse later?

Why Separates people who think in terms of consequences from people who think in terms of tasks. The database model and the auth approach should appear in the answer.

Red flag "Nothing is hard to change." Some things always are, and the data model is usually one of them.

Process and communication (32–36)

How you find out something is wrong — early, rather than at the end.

32 What will I see, and when?

Why You want a working, deployed URL you can open yourself, on a stated rhythm. Screenshots and demo videos can be produced from work that does not function.

Red flag A single demo at the end, or progress reported only as percentages.

33 How do I reach you, and what is a realistic response time?

Why Set the expectation before you need it. Both directions — they will need answers from you fast too.

Red flag A ticketing system with no named human, or a promise of instant availability that nobody can keep.

34 When we disagree about whether something is a bug or a new request, how is that resolved?

Why This argument happens on nearly every project. Deciding the mechanism in advance costs nothing; deciding it during the argument costs the relationship.

Red flag No mechanism, or unilateral classification by the provider.

35 What decisions will you need from me, and how quickly?

Why On a fast project, a decision that waits three days is a project that slips three days. You need to know what will land on you and when.

Red flag No expectation set, which usually means the delay will later be attributed to you.

36 Where does the work live so that I can see it as it happens?

Why Repository access from day one is normal, cheap, and the strongest possible transparency signal.

Red flag Access granted only at handover.

Quality, testing and security (37–42)

The work you are paying for that you will never see — until it fails.

37 What is covered by automated tests, and what is not?

Why The honest answer is never "everything". You want tests concentrated where failure is expensive: payments, authentication, anything that writes or deletes data.

Red flag "Full test coverage" on a one-week build, or no tests at all on anything touching money.

38 How are passwords, keys and credentials handled?

Why You cannot audit this yourself, but you can hear the difference between a considered answer and an improvised one.

Red flag Secrets in the repository, credentials shared over email or chat, or no mention of a secret manager or environment variables.

39 What is the backup plan, and have you ever tested a restore?

Why The second half is the real question. Untested backups fail at exactly the moment they are needed, and most people have never tried.

Red flag Backups exist but have never been restored, or backups stored only in the same place as the data.

40 When something breaks in production, how do you find out?

Why The answer should be a monitoring or error-tracking tool that alerts someone. The alternative is that your users are your monitoring.

Red flag "The client tells us."

41 How is personal data handled, and where is it stored?

Why If you serve EU or UK users, this is a legal obligation that lands on you, not on your provider. Data location, retention and deletion should all have answers.

Red flag No idea where the database physically is, or no ability to delete a user's data on request.

42 What is your definition of done for a feature?

Why Done can mean "it works on my machine" or "it is deployed, tested, monitored and documented". The gap between those is weeks.

Red flag Done means the code is written. Nothing is finished until it is running in production.

Launch, handover and life after (43–47)

What happens on the day the project ends, and the month after.

43 Walk me through launch day. What actually happens?

Why A specific answer — domain, DNS, deployment, monitoring, a check of the live flows — shows they have done it. A vague one shows they have not, recently.

Red flag Launch treated as a formality, or as your responsibility.

44 What documentation do I receive?

Why The minimum is a README that lets another developer run the project locally, plus documented environment variables and an architecture summary.

Red flag None, or documentation offered as a paid extra.

45 Will you walk my next developer through the codebase?

Why Tests whether they are planning for your independence or for your dependence. An hour of handover is cheap and enormously valuable.

Red flag Reluctance, or a large fee for a conversation.

46 What are the three things most likely to break in the first month?

Why Everyone who has shipped knows the answer for their own work. It is also the most generous thing a provider can tell you, so it doubles as a character test.

Red flag "Nothing." Something always breaks in the first month.

47 If this were your own product, what would you do differently?

Why The closing question. It gives them permission to tell you the truth about your plan, and the answer is often worth more than the rest of the call.

Red flag Flattery. Anyone with real experience has an opinion about what you should change.

Questions

I am not technical. Can I really evaluate these answers?

Yes — because the questions are designed so that the shape of the answer carries the signal, not its technical content. Specific beats reassuring, volunteered downsides beat confidence, and "I would need to check" beats a fast wrong answer. You are assessing how someone thinks, which you do every day in every other part of your business.

Which questions matter most if I only ask a few?

Numbers 1 (describe my product back to me), 9 (who pays for an over-run), 11 (what is not included), 20 (what stops working if I leave) and 21 (who writes the code). Those five surface most of the risk in about ten minutes.

Is it rude to ask a provider all of this?

No, and the reaction is itself data. You are about to hand someone a significant amount of money for something you cannot inspect. Any professional expects diligence; only someone with a problem finds it offensive.

What should a V1 cost, so I know if a quote is reasonable?

Market ranges in 2026: roughly \$20,000–50,000 with a senior freelancer, \$60,000–250,000 with a classic agency, and \$17,000 fixed for a Khufu Sprint V1 delivered in 7 days. A quote far below the range usually means no-code, offshore juniors, or a scope that will grow after signature.

How does Khufu answer these questions?

Fixed price agreed before the work starts, and an over-run is the agency's cost. One person to talk to, who decides, from the first exchange to the handover. The repository sits on your organisation from day one with full history, and every infrastructure account is in your name. If the relationship ended tomorrow, nothing would stop working except further changes.

Ask me these 47 questions

That is a genuine invitation — the list was written to be answered. Sprint V1: a SaaS or mobile app scoped, built and shipped in 7 days for a fixed \$17,000, with the repository yours from day one.



Message me on WhatsApp

Tap the button — it opens a chat with +971 50 365 1761, message already written. I read every one myself.

Talk to the person who would build it

Adrien De Coster, founder of Khufu — an AI-native product agency in Dubai. I write the code on every sprint, so the conversation about your product is with the person who builds it.

- Email: hello@khufu.io
- Site: <https://khufu.io>
- LinkedIn: <https://www.linkedin.com/in/adriendecoster/>
- X: https://x.com/adriendecoster_

Questions about anything in this guide are welcome even if you never hire me — the fastest way to get one answered is a direct message.